Les tests fonctionnels avec Selenium WebDriver

Tests end-to-end

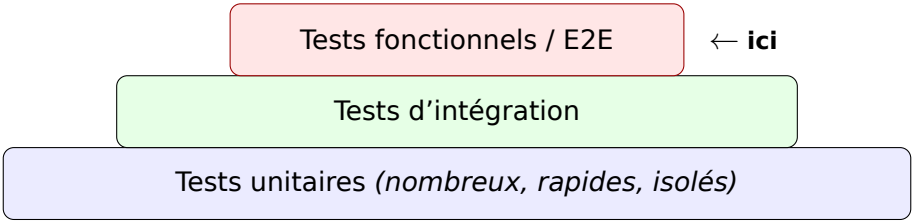
Wissem Inoubli

7 octobre 2026

Pourquoi des tests fonctionnels automatisés ?

Cas d'usage. Vous venez de modifier le formulaire de connexion d'un site. Comment être sûr, en quelques secondes, que le parcours client fonctionne toujours sans recliquer vous-même chaque champ ?

- Un test fonctionnel vérifie le comportement **du point de vue de l'utilisateur**
- Sans automatisation, quelqu'un doit rejouer le même scénario à la main à chaque changement de code

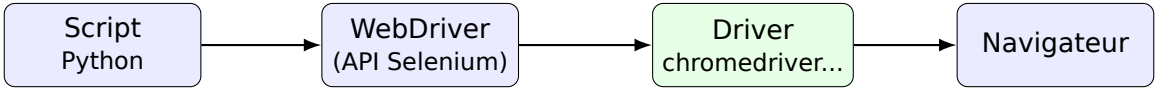


Qu’est-ce que Selenium WebDriver ?

- **Selenium** : projet open source qui pilote un navigateur depuis du code le doyen des outils de ce type, standard de facto depuis le milieu des années 2000
- **WebDriver** : l’API qui pilote directement le navigateur, par opposition à Selenium IDE (enregistreur) ou Selenium Grid (exécution distribuée)
- Repose sur le protocole standardisé **W3C WebDriver** : chaque navigateur a son propre driver (`chromedriver`, `geckodriver`…) qui traduit les commandes Selenium en actions natives

À utiliser quand vous avez une application **web** et voulez piloter un **vrai navigateur** Selenium reste la référence pour les environnements où seul un driver W3C standard est disponible.

Architecture : du script au navigateur



Le script appelle l'API WebDriver en Python ; celle-ci parle au driver du navigateur via le protocole standardisé **W3C WebDriver** ; le driver pilote enfin le navigateur nativement. Chaque commande (`click()`, `send_keys()`...) refait cet aller-retour complet à la différence de Playwright, qui parle un protocole plus direct (CDP) à Chromium.

Installation Selenium Manager

```
pip install selenium
```

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://www.iut-lens.univ-artois.fr/")
print(driver.title)
driver.quit()
```

Depuis Selenium 4.6, un **Selenium Manager** intégré télécharge et associe automatiquement le bon driver plus besoin de le télécharger à la main ni de faire correspondre sa version à celle du navigateur. `webdriver.Chrome()` sans argument suffit dans le cas standard.

Locators les stratégies de By

À utiliser dès qu'une action doit cibler un élément précis c'est la première chose à écrire dans un test.

Stratégie	Cible
`By.ID`	attribut `id`
`By.NAME`	attribut `name`
`By.CLASS_NAME`	attribut `class`
`By.CSS_SELECTOR`	un sélecteur CSS
`By.XPATH`	une expression XPath
`By.TAG_NAME`	le nom de balise (`a`, `input`…)
`By.LINK_TEXT`	le texte exact d'un lien
`By.PARTIAL_LINK_TEXT`	un texte partiel de lien

Actions interagir avec la page

À utiliser pour reproduire un parcours utilisateur, étape par étape.

Fonction	Effet
`driver.find_element(by, valeur)`	localiser un élément
`element.click()`	cliquer
`element.send_keys(texte)`	saisir du texte
`element.clear()`	vider un champ
`element.submit()`	soumettre le formulaire
`Select(element).select_by_visible_text(t)`	choisir dans une liste

`Select` vient de `selenium.webdriver.support.ui` une classe dédiée aux listes `<select>`, sans équivalent direct pour les cases à cocher ou boutons radio : on les cible et on les `click()` comme n'importe quel élément.

Locators relatifs (Selenium 4)

```python
from selenium.webdriver.support.relative_locator import locate_with

mot_de_passe = driver.find_element(By.ID, "password")
email = driver.find_element(
    locate_with(By.TAG_NAME, "input").above(mot_de_passe)
)
```

Exemple concret. Un champ sans id stable, mais toujours positionné juste au-dessus du mot de passe : `above()`, `below()`, `to_left_of()`, `to_right_of()` et `near()` le retrouvent par position plutôt que par sélecteur.

Attente implicite vs explicite

Pourquoi ça compte. Selenium **n'attend pas** automatiquement qu'un élément soit prêt avant d'agir.

- **Attente implicite** un réglage *global* :

```
driver.implicitly_wait(10)
```

attend jusqu'à 10 s avant de lever `NoSuchElementException`, mais ne vérifie que la présence dans le DOM pas la visibilité ni le caractère cliquable.

- **Attente explicite** une condition *précise*, au bon endroit du test : un `WebDriverWait` associé à une `expected_condition`.

Attente explicite exemple

```python
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

wait = WebDriverWait(driver, 10)
bouton = wait.until(EC.element_to_be_clickable((By.ID, "submit-btn")))
bouton.click()
```

Conditions courantes : `presence_of_element_located`, `visibility_of_element_located`, `element_to_be_clickable`, `text_to_be_present_in_element`, `alert_is_present`.

Assertions

À utiliser pour vérifier un **résultat**, pas seulement qu'une action s'est exécutée sans erreur.

Selenium n'a pas d'équivalent intégré au `expect()` de Playwright : on utilise `assert` directement sur les propriétés de l'élément.

`element.text`	contenu texte
`element.get_attribute(nom)`	valeur d'un attribut
`element.is_displayed()`	est visible
`element.is_enabled()`	est actionnable
`element.is_selected()`	case/radio coché

Exemple concret. Une assertion lancée juste après une attente *implicite* peut échouer même si l'élément finit par être correct : elle ne réessaie pas d'elle-même. Faites précéder toute assertion d'une attente *explicite* sur la bonne condition.

Structurer avec unittest

```python
import unittest
from selenium import webdriver

class LoginTest(unittest.TestCase):
    def setUp(self):
        self.driver = webdriver.Chrome()

    def test_login(self):
        self.driver.get("https://example.com/login")
        # ...

    def tearDown(self):
        self.driver.quit()
```

`setUp()` / `tearDown()` s'exécutent avant / après **chaque** méthode `test_*` de la classe la manière historique d'isoler les tests Selenium.

Exemple

```python
from selenium.webdriver.common.by import By

class LoginPage:
    URL = "https://example.com/login"

    def __init__(self, driver):
        self.driver = driver

    def open(self):
        self.driver.get(self.URL)

    def login(self, user, pwd):
        self.driver.find_element(By.ID, "user-name").send_keys(user)
        self.driver.find_element(By.ID, "password").send_keys(pwd)
        self.driver.find_element(By.ID, "login-button").click()

page = LoginPage(driver); page.open(); page.login("demo", "demo")
```

Boîtes de dialogue (alerts)

```python
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver.find_element(By.ID, "supprimer-btn").click()

WebDriverWait(driver, 10).until(EC.alert_is_present())
alert = driver.switch_to.alert
alert.accept()   # ou alert.dismiss(), alert.send_keys(...)
```

Fenêtres et onglets multiples

```
fenetre_principale = driver.current_window_handle
driver.find_element(By.ID, "ouvrir-onglet").click()

for handle in driver.window_handles:
    if handle != fenetre_principale:
        nouvelle = handle

driver.switch_to.window(nouvelle)
print(driver.title)

driver.switch_to.window(fenetre_principale)
```

`driver.window_handles` liste les onglets ouverts ; `switch_to.window()` bascule explicitement vers une autre fenêtre.

Captures d’écran

```
driver.save_screenshot("erreur.png")
element.screenshot("element.png")
```

À utiliser pour comprendre visuellement un échec survenu en CI.